

Programmering

Innehållsförteckning

- [C/C++ Strängar](#)
 - [Viktigt att tänka på!](#)
 - [Text strängar](#)
 - [Funktioner för att manipulera strängar](#)
 - [Sträng hantering i C++ mha STL](#)
 - [Hur gör man egentligen för att kolla en enskild bit i ett heltal?](#)
 - [Pekar, vad är det?](#)
 - [Skillnaden mellan *void main\(\)* och *void main\(void\)*?](#)
 - [Jag har ett program som inkluderar iostream-filen, men får ett meddelande om att den inte finns, varför?](#)
 - [Vad är en kompilator \(compiler\)](#)
 - [Vad är en länkare \(linker\)](#)
 - [Vad är objekt-file](#)
 - [Vad är lib-file](#)
 - [Vad är skillnaden på att definiera och att deklarera en funktion/metod?](#)
 - [Vad är .h-filer?](#)
 - [Hur gör jag en .h-fil?](#)
 - [Var kan jag på tag på xxx.h?](#)
 - [Jag har xxx.h men kompilatorn klagar ändå, vad är fel?](#)
-

C/C++ Strängar

Viktigt att tänka på!

1. Definiera eller allokerar char vektorn tillräckligt stor så att all text inklusive nolltecknet får plats.
2. När du kopierar in text i vektorn (strcpy) eller lägger till text (strcat) att vektorn du kopierar till är tillräckligt stor. Ingen av dessa funktioner kontrollerar att vektorn är tillräckligt stor, vilket kan medföra att andra variabler skrivs över och om det vill sig illa att din programkod skrivs över!

Text strängar

C har ingen variabeltyp som man kan lagra textsträngar i, det man får göra är att definiera upp en vector av type char som är tillräckligt stor, och sedan kopiera in textsträngen till vektor. Och **detta är viktigt!** det sista tecknet i variabel är "nulltecknet", (Tecknet har ASCII värde = 0).

Följande program visar hur man skapar en vektor av type char och kopierar in en text i vektorn:

```
#include

main(void) {
//
// Skapar en vektor av type char.
char Text[20];
//
// Detta är det enda fallet man kan skriva 'variabel="textsträng"' om man vill
// initiera variabel.
```

```

    char Text2[20]="GunnarD";
//
// Räkna variabel för for satserna.
    int i;

//
// Följande rad fungerar inte! Ta gärna bort kommentartecknena och se
// vad som händer!!
// Text="GunnarD";
//
// Så här kopierar man in text i en "text variabel"
    strcpy(Text, "GunnarD");
//
// Skriv ut Text strängen med tillhörande ASCII kod (i hexformat)
    for(i=0;i<20;i++) {
        printf("%2c ", Text[i]);
        if(Text[i]==0)
            break;
    }
    printf("\n");
    for(i=0;i<20;i++) {
        printf("%02X ", Text[i]);
        if(Text[i]==0)
            break;
    }
    printf("\n");
}

```

Funktioner för att manipulera strängar

Dessa funktioner finns det för att manipulera "sträng variabler" (eg. vektorer av char's) för att kunna använda dessa funktioner så måste man inkludera string.h:

Funktion	Förklaring	Exempel
strcpy(char *text1, char *text2)	Kopierar char's från text2 till text1 till och med att nolltecknet på träffas.	str_strcpy.c
strncpy(char *text1, char *text2, int antal)	Kopierar char's från text2 till text1 till och med att nolltecknet på träffas eller att "antal" tecken har kopierats. I sista fallet så kopieras inte nolltecknet med, man måste sätta in det manuellt!	str_strncpy.c
strcat(char *text1, char *text2)	Kopierar in text2 i text1 med början på första nolltecknet. Läger till texten från text2 till text1	str_strcat.c
strncat(char *text1, char *text2, int antal)	Kopierar in text2 i text1 med början på första nolltecknet. Läger till texten från text2 till text1, dock max "antal" tecken!	str_strncat.c
strcmp(char *text1, char *text2)	Jämför strängarna text1 och text2, returnerar 0 om dom är lika, minustal om text1 är mindre än text2 och positivt tal om text1 är större än text2.	str_strcmp.c
strncmp(char *text1, char *text2, int antal)	Jämför strängarna text1 och text2, returnerar 0 om dom är lika, minustal om text1 är mindre än text2 och positivt tal om text1 är större än text2. Dock jämförs max "antal" tecken!	str_strncmp.c
strlen(char *text1)	Returnerar strängens längd, MEN inkluderar inte nolltecknet, så om du vill använda detta för att ex. alloka utrymme så måste du lägga på 1 för nolltecknet!	str_strlen.c

Sträng hantering i C++ mha STL

Skrivet av Jens Bäckman (Chainsaw) 19990811.

Stränghantering i C++ har förenklats avsevärt sedan STL, Standard Template Library, dök upp. Det finns nu inbakat i alla kompilatorer som följer (eller vill följa) ANSI-standarderna. För att få tillgång till dessa roliga tingestar skriver du:

```
#include <string>
```

Notera att det inte ska vara något H i slutet. Här är ett program som gör exakt samma sak som det ovanstående exemplet för C.

```
#include <string>
#include <iostream.h>

main(void) {
    // Skapa en sträng med namnet Text
    string Text;

    // Det går alldeles utmärkt att initialisera en sträng direkt när
    // den skapas
    string Text2 = "Chainsaw";

    // Vi kan ändra värdet direkt även efter det att vi skapat en
    // variabel
    Text = "GunnarD";

    // För att kopiera Text2 till Text skriver man logiskt nog:
    Text = Text2;

    // Skriv ut Text-strängen med tillhörande ASCII kod (i hexformat)
    int i;
    cout << " ";
    for(i = 0; i < 20; i++) {
        cout << Text[i] << " ";
        if(Text[i] == 0) break;
    }
    cout << endl;
    cout << hex;    // Byt till hex-läge
    for(i = 0; i < 20; i++) {
        cout << (int)Text[i] << " ";
        if(Text[i] == 0) break;
    }
    cout << endl;
}
```

Hur gör man egentligen för att kolla en enskild bit i ett heltal?

```
if(t & (1 << b)) {
    // utförs om bit nummer b är 1 i tal t. (Bit nummer 0 är lägsta biten i talet.)
}
```

Pekar, vad är det?

Följande är kopierat från Passagens C/C++ konferens, beskriver vad en pekar är och vad man kan ha dom till! :-))

Föreställ dig en gigantisk neonskylt med texten "HAN STÅR DÄR". Ungefär det är en pekare. Det talar bara om var någon står, men inte vem det är eller vad han/hon/den/det heter. Vettiga användningsområden? Föreställ dig en array som tar upp 2MB minne. Istället för att kopiera hela härligheten innan du sänder den till en funktion, så tar du fram din gigantiska neonskylt och riktar den mot din array innan du kastar in den i funktionen.

Skrivet av Chainsaw 19990714

Skillnaden mellan *void main()* och *void main(void)*?

`void main()` I C betyder det att `main` inte returnerar någonting, och att antal parametrar inte är känt (allt från 0 och uppåt, samma som `void main(...)`);

I C++ betyder det att `main` inte returnerar någonting och att det inte skall vara några parametrar/argument.

`void main (void)` Betyder samma i C och C++, returnerar ingenting och inga parametrar/argument.

Jag har ett program som inkluderar *iostream*-filen, men får ett meddelande om att den inte finns, varför?

Du försöker antagligen att kompilera programmet som ett C program och inte som ett C++ program. Om du använder `gcc` (eller `djgpp`) så använd `gcc` för att kompilera ett C program och `g++` för att kompilera ett C++ program. En del kompilatorer är "smarta" och kollar om filen slutar med `.c` så används C kompilatorn och slutar filen med `.cpp` (eller `.cc`) så används C++ kompilatorn.

Vad är en kompilator (compiler)

En dator kan inte förstå ett C/C++ - program så som det ser ut när du har skrivit det. Därför måste det översättas till datorns eget språk (kallas vanligtvis maskin-språk). Det är kompilatorns uppgift. Det tar dina instruktioner och översätter till datorns egna instruktioner. När kompilatorn är klar med detta mellanlagras normalt dessa i en s.k. objekt-fil. (Dessa har filändelsen `.o` eller `.obj`.)

Anledningen till detta - till synes onödiga - mellansteg är för att man ska kunna göra stora program bit för bit. Kompilering av stora program tar jättemycket tid och därför vill man kunna kompilera delar av det. Dessa objektfiler är inte färdiga program utan det är länkarens uppgift att sammanfoga - att "länka" ihop - dessa till ett färdigt program. Normalt görs detta automatiskt och du märker inte att det är skillnad på kompilator och länkare utom i ett visst fall vilket är när du deklarerat något utan att definiera det. (Se mer om det nedan).

Skrivet av: Michael Lundahl

Vad är en länkare (linker)

Kompilator och länkare följer alltid med varandra eftersom de är i princip värdelösa om de inte finns tillsammans. Se [Vad är en kompilator \(compiler\)](#) ovanför vad länkaren gör.

Skrivet av: Michael Lundahl

Vad är objekt-filer

Se [Vad är en kompilator \(compiler\)](#) ovanför vad objekt filer är och när dom skapas (och varför).

Vad är lib-filer

Lib-filer - s.k. biblioteks-filer (library files) - är många objekt-filer ihopslagna till en enda stor fil. Detta är för att man ska kunna skapa stora bibliotek med nyttiga saker som andra sedan ska kunna använda utan att de ska behöva kompilera om alltihop själva. Till varje lib-fil finns alltid en s.k. `.h`-fil (header file) som berättar vilka funktioner/metoder/variabler som finns inbakade i lib-filen.

Skrivet av: Michael Lundahl

Vad är skillnaden på att definera och att deklarera en funktion/metod?

Deklarerad/deklaration betyder att du har i programmet talat om "Det finns en funktion/metod/variabel som heter xxx". Funktions/metod-deklarationer innehåller **INTE** programkod som gör något. Den koden skrivs i funktions/metod-definitionerna.

Definerad betyder att funktioner/metoder har programkod som gör något. Om en funktion/metod har deklarerats **MÅSTE** den också definieras. Dock är det **INTE** nödvändigt att alla funktioner/metoder deklarerats innan de definieras. En definition är nämligen en deklaration i sig.

Exempel på en deklaration:

```
void enFunktion(int enParameter);
```

Exempel på motsvarande definition:

```
void enFunktion(int enParameter){  
    for(int i = 0; i < enParameter; i++){  
        printf("Denna rad har nu skrivits %d gånger\n", i);  
    }  
}
```

Skrivet av: Michael Lundahl

Vad är .h-filer?

Headerfiler (filer som slutar på .h .hh eller .hpp) är till för att underlätta det jag nämnde i föregående tidigare, d.v.s. möjligheten att kompilera program bit för bit och sedan eventuellt baka ihop dem i biblioteksfiler. När du vill använda en viss funktion/metod/varibel - vare sig du skriver den själv eller den finns färdigkompilerad - måste denna vara deklarerad eller definierad. C och C++ kompilatorer tillåter att du enbart deklarerar funktioner/metoder utan att definiera dem. Kompilatorn nöjer nämligen sig med att veta "Ok, någonstans bland programmets alla filer finns en funktion som heter xxx". Det är just .h-filernas uppgift att ge denna information. .h-filerna talar om för kompilatorn "När det är dags för länkning så kommer funktionerna xxx, yyy o.s.v. att finnas så det är ok att användaren använder dem nu". h-filer skall **bara** innehålla deklarationer men **aldrig** definitioner.

(En header fil kan innehålla definitioner, speciellt vanligt är det när det gäller makron till preprocessorn. Men ytterst sällan definitioner av funktioner . GunnarD)

Skrivet av: Michael Lundahl

Hur gör jag en .h-fil?

.h-filer har du endast användning av om ditt program är uppdelat i många filer, annars är det bara onödigt jobb att skriva en .h-fil. *(Men det är vanliga text filer, GunnarD)*

Skrivet av: Michael Lundahl

Var kan jag på tag på xxx.h?

Kompilatorn klarar sig med att de funktioner/metoder du vill använda enbart är deklarerade men däremot kräver **länkaren** att dessa också är definierade och/eller färdigkompilerade. Därför räcker det inte med att du har fått tag på en viss .h-fil... du måste också få tag på

- 1) Motsvarande definitioner (d.v.s programkod) eller...
- 2) Objektfiler som innehåller de färdigkompileerade funktionerna/metoderna. (Eller .lib filen som innehåller objektfilerna. GunnarD)

Har du inte någon av dessa kommer du inte att kunna länka ditt program.

Skrivet av: Michael Lundahl

Jag har xxx.h men kompilatorn klagar ändå, vad är fel?

Antag t.ex. att du har fått tag på windows.h. Vad finns då i den filen? Jo där finns massor av deklarationer... men inte **en enda definition**. Vad göra? Jo du måste ha tag på en fil som heter windows.lib också för där finns alla de funktioner/metoder som nämns i windows.h färdigkompileerade och klara. Har du skaffat dig en .h-fil någon annanstans ifrån **MÅSTE** du också skaffa dig tillhörande .lib-fil, annars kommer länkaren att bråka och du kommer inte att få ett körbart program. Därför är det oftast helt lönlöst att fråga efter en .h-fil. Det du istället skall göra i den situationen är att skaffa dig en kompilator/länkare som redan har de .h-filer och .lib-filer du behöver.

Skrivet av: Michael Lundahl